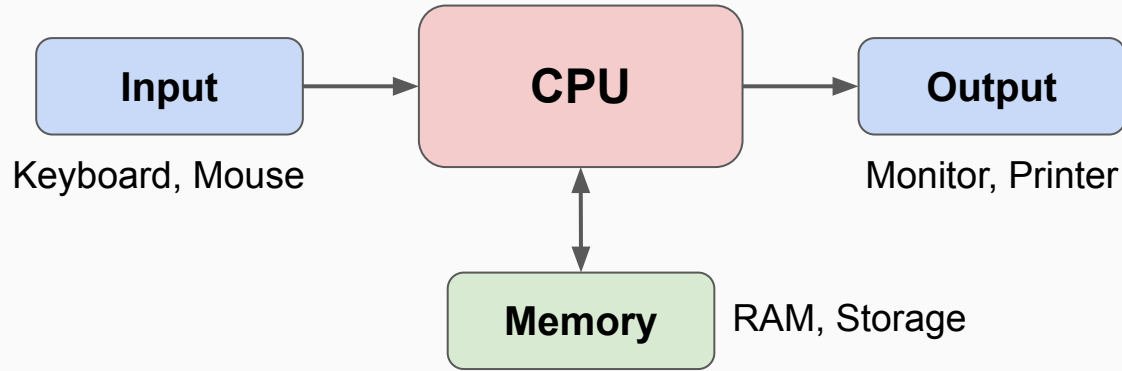


Introduction

Mahir Labib Dihan
Lecturer, CSE, BUET



How a Digital Computer Works



- **Input:** Receives data (e.g., from keyboard)
- **CPU (Central Processing Unit):** The "brain" that processes data.
- **Memory:** Stores data and programs temporarily (RAM) or permanently (Disk).
- **Output:** Displays results

Computers Are Powerful but Dumb

A Computer Only Follows Instructions

- Computers cannot think.
- They only execute instructions exactly as given.
- Every task must be described step by step.

Example: Making tea requires instructions:

1. Boil water
2. Add tea leaves
3. Pour into cup

Computers work the same way!

How Does a Computer Understand Instructions?

The Language of Computers

Computers understand only two symbols:

- 0
- 1

Example:

10110000 01100001

This is called **machine language** (binary).

The Problem with Machine Language

Programming Directly in Binary Is Difficult!

Imagine writing:

```
10110000 01100001  
10110010 00000001  
11001101 00100001
```

Problems:

- Hard to read
- Hard to remember
- Easy to make mistakes
- Difficult to debug

A Better Solution – Programming Languages

Human-Friendly Languages

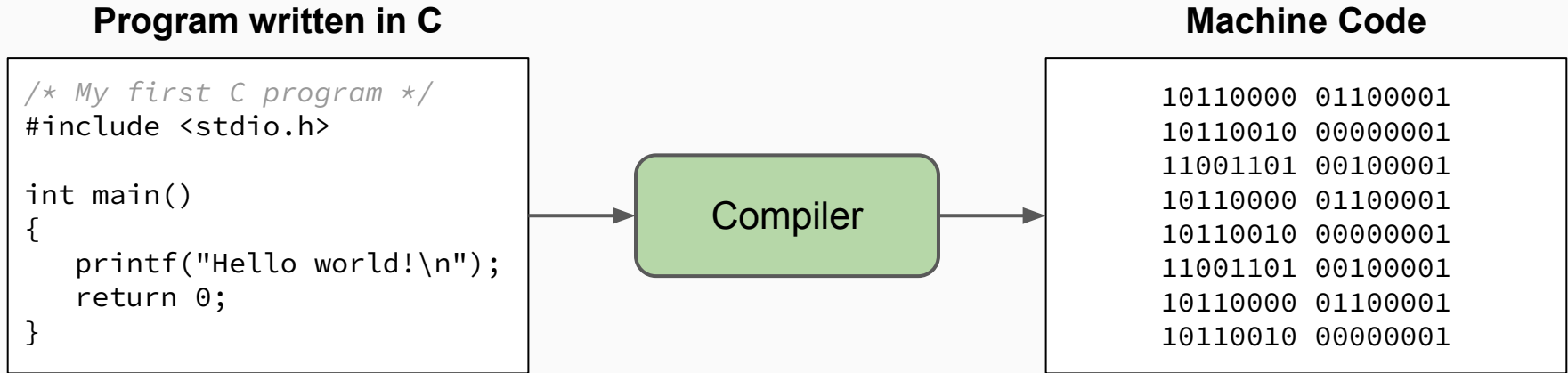
Instead of binary, we can write them in *C language*:

```
printf("Hello, World!");
```

Advantages:

- Easier to read
- Easier to write
- Easier to maintain
- Easier to find errors

Who Translates Our Program?



The compiler acts as a translator between humans and computers.

Why Learn C?

The Importance of C

- One of the most influential programming languages.
- Fast and efficient.
- Used in operating systems, embedded systems, and software development.
- Helps understand how computers work internally.
- Foundation for many modern languages.

Welcome to Programming!

What is a Computer Program?

A computer program is a sequence of instructions written to perform a specified task with a computer.

C programming language

- Structured and disciplined approach to program design
- Story → Paragraph → Sentence
- Program → Function → Statement

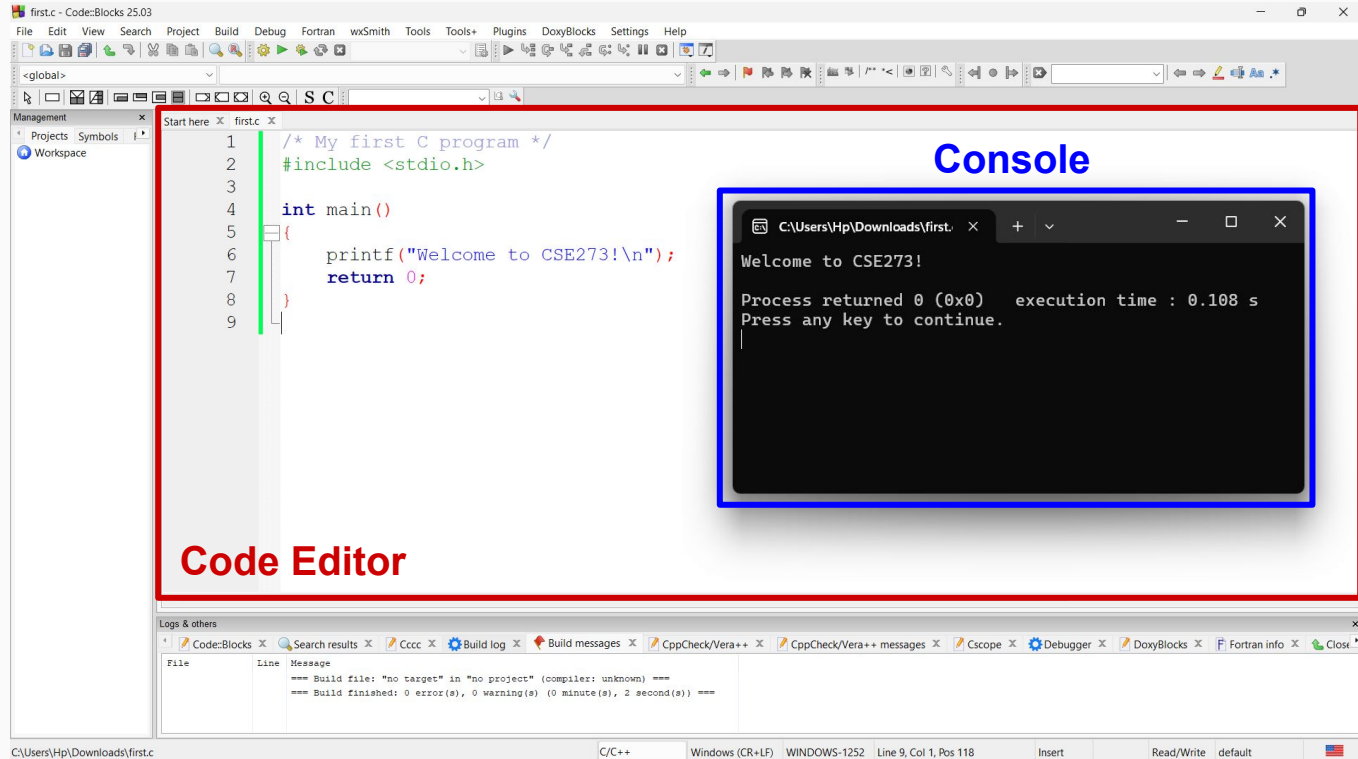
Your First C Program

```
/* My first C program */
#include <stdio.h>

int main()
{
    printf("Welcome to CSE273!\n");
    return 0;
}
```

- Save a source code with extension “c”. (Ex: **first.c**)
- Run the program.
- **Output of the program:** Welcome to CSE273!

Code::Blocks



Your First C Program: Comments

```
/* My first C program */ ←  
#include <stdio.h>  
  
int main()  
{  
    printf("Welcome to CSE273!\n"); // Output ←  
    return 0;  
}
```

- Notes for humans, ignored by the computer.
- Two types of comment
 - Text surrounded by `/*` and `*/`
 - Text preceded by `//`

Your First C Program: #include <stdio.h>

```
/* My first C program */  
#include <stdio.h>   
  
int main()  
{  
    printf("Welcome to CSE273!\n"); // Output  
    return 0;  
}
```

- #include <stdio.h>
 - Preprocessor directive - tells computer to load contents of a certain file
 - <stdio.h> allows standard input/output operations

Your First C Program: `int main()`

```
/* My first C program */
#include <stdio.h>

int main() ←
{
    printf("Welcome to CSE273!\n"); // Output
    return 0;
}
```

- `int main()`
 - C programs contain one or more functions, exactly one of which must be **main**
 - Parenthesis used to indicate a function
 - **int** means that main "returns" an integer value
 - Braces indicate a block
 - The bodies of all functions must be contained in braces

Your First C Program: printf

```
/* My first C program */
#include <stdio.h>

int main()
{
    printf("Welcome to CSE273!\n"); ←
    return 0;
}
```

- `printf("Welcome to C!\n");`
 - Instructs computer to perform an action
 - Specifically, prints string of characters within quotes
 - Entire line called a **statement**
 - All statements must end with a semicolon

Your First C Program: return 0;

```
/* My first C program */  
#include <stdio.h>  
  
int main()  
{  
    printf("Welcome to CSE273!\n"); // Output  
    return 0;   
}
```

- `return 0;`
 - A way to exit a function
 - `return 0`, in this case, means that the program terminated normally

Resources for Learning

Recommended Textbooks:

- Teach Yourself C by Herbert Schildt
- The C Programming Language by Kernighan & Ritchie
- Numerical Methods for Engineers by Chapra & Canale

Online Resources:

- GeeksforGeeks - Detailed explanations
- cprogramming.com - Tutorials and examples
- Programiz - Interactive learning
- W3Schools - Reference guide
- YouTube - Video tutorial

Acknowledgement

- Dr. Ashikur Rahman, Professor, CSE, BUET
- Mohammad Sadat Hossain, Lecturer, CSE, BUET